

SOFTENG 752

Formal Specification and Design

Assignment #1

Library Management System

Name: Enyang Zhang

UPI: ezha812

ID: 721492553

Assignment partner: Jennifer Xie

UPI: yxie614

ID: 556477746

1. Introduction

This report presents a B-Method specification and ProB-based verification of a Library Management System (LMS). At a high level, an LMS registers titles and physical copies, supports reservations and loans, tracks per-title inventory and per-copy status, maintains FIFO waiting lists when stock is exhausted, and any changes to the catalogue, inventory, or membership must be made by an administrator.

Standard system-design material on library systems (e.g., GeeksforGeeks, “System Design for Library Management”) was consulted as a modelling reference. The core lending workflow was selected as the focus, and fines, due dates, and multi-branch operations were excluded.

This report outlines the key system requirements that ground our model; explains the B specification—its sets, variables, invariants, and representative operations—together with the rationale behind the main design choices; and presents the ProB-based verification, including the methods applied (model checking, assertions, LTL), the configurations used, and the principal outcomes. It closes with a brief discussion of model completeness, known limitations, and directions for improvement.

2. System Requirements

This section states the system requirements for the LMS. The system’s functions (what the system must do), behaviours (rules and constraints it must obey), and interactions (how administrators and members use the system, and how internal components work together) are described. The requirements were presented as numbered Functional Requirements (FR) and were grouped by domain: Catalogue & Titles (C), Inventory & Copies (I), Reservations & Waiting (R), Loans (L), Membership & Limits (M), and Administration & Access (A). Each requirement was thereafter referenced by its short code (e.g., FR-C1).

2.1 Functional Requirements

Catalogue & Titles (C)

- The system shall allow an administrator to register a new title and initialise its stock to zero with an empty waiting list. (FR-C1)
- The system shall allow deletion of a title only by an administrator, and only when (i) the title’s stock is zero, (ii) no copy is catalogued to that title, and (iii) the title’s waiting list is empty. (FR-C2)

Inventory & Copies (I)

- The system shall allow an administrator to add a new physical copy to a title, increasing that title’s stock by one, up to a fixed capacity limit. (FR-I1)
- The system shall allow an administrator to remove a physical copy only if the copy is neither reserved nor borrowed; the presence of a waiting list is not an additional restriction, since any available copy is immediately reserved for the queue head by the system and thus not removable. (FR-I2)
- The system shall display per-copy status (available/booked/borrowed) and per-title stock (count of available + borrowed copies). (FR-I3)

Reservations & Waiting (R)

- The system shall allow a member to reserve an available copy when the borrow limit is not reached, changing its status to booked without changing the title's stock. Other members will not be allowed to borrow the copy. (FR-R1)
- When no copy of a title is available, the system shall allow a member to join that title's waiting list at most once; only registered members may appear in waiting lists. (FR-R2)
- When stock increases (a new copy is added) or a borrowed copy is returned, if the waiting list is non-empty, the system shall automatically assign that copy to the head and set its status to booked, removing the head from the list. (FR-R3)
- Members in waiting lists shall count towards the borrow limit, which ensures that members will have the capacity to hold the waiting books. (FR-R4)
- The system shall allow a member to withdraw a reservation on a copy booked for that member. (FR-R5)
- The system shall allow a member to remove themselves from a title's waiting list. (FR-R6)

Loans (L)

- The system shall allow a member to borrow a copy that is available, or to borrow a copy that is reserved for the same member; borrowing shall decrease the title's stock by one and set the copy to borrowed. (FR-L1)
- The system shall allow a member to return a borrowed copy; returning shall increase the title's stock by one and set the copy to available. (FR-L2)
- The system shall prevent borrowing when doing so would exceed the member's borrow limit. (FR-L3)

Membership & Limits (M)

- The system shall enforce a per-member borrow limit that counts among held copies, booked copies and the number of distinct titles for which the member is in a waiting list. (FR-M1)
- The system shall prevent a member from joining a waiting list if there is an available copy of that title or if the member already holds a copy of that title. (FR-M2)
- The system shall allow deletion of a member only by an administrator, and only when the member holds no copies in any state (neither booked nor borrowed) and appears on no waiting list. (FR-M3)

Administration & Access (A)

- State-changing actions on titles, copies, inventory, and membership shall be restricted to administrators. (FR-A1)
- Within the model's scope, creation and deletion of administrator records were permitted without additional privilege checks; in practice, these operations would be reserved for a higher-privilege role. (FR-A2)

2.2 Non-Functional/Scope

- Overdue timers, fines, renewals, and inter-library loans were not modelled.
- Fixed bounds were used: the borrow limit was set to 2 and the per-title copy capacity (limit) was set to 4; these bounds were used by ProB to keep the state space finite.

3. B Design Model and Verification

3.1 ProB Configuration

Because NAT was used, MAXINT was set above 4 (e.g., 8–10) and DEFAULT_SETSIZE was set to 2 to bound deferred sets (BOOK, BOOK_NAME, MEMBER, ADMIN); these were ProB analysis settings and did not change the model's abstract semantics.

3.2 Modelling Scope and Abstractions

ProB was used for analysis; finite domains and lightweight abstractions were adopted; the system includes catalogues and titles, inventory and copies, reservations and waiting lists, borrowing and returning, and administrator/member access.

Fixed bounds (limit=4, borrow_limit=2) and the single-branch assumption were applied to keep the state space finite and checkable in ProB.

3.3 SETS

Sets BOOK, BOOK_NAME, MEMBER, and ADMIN were introduced as abstract identifiers for copies, titles, members, and administrators to keep domains implementation-independent (and merely bounded in ProB); the small, fixed lifecycle BOOK_STATUS={available, booked, borrowed} was enumerated to capture the only allowed per-copy states.

3.4 CONSTANTS & PROPERTIES

The constants limit=4 (per-title capacity) and borrow_limit=2 (per-member quota) were set; typing and related conditions were stated in PROPERTIES.

3.5 VARIABLES

The variables were designed as follows: books (copies), book_name (titles), book_catalogue (relation of copy and title), book_status (copy status), inventory (title stock), waiting (title waiting list), own_book (relation of member and copy), members, and admins. Their types and intended use were checked by the invariant.

3.6 INVARIANT

The invariant first stated basic typing: the sets of administrators, members, copies, and titles were kept consistent with their intended domains; each copy, when catalogued, was linked to exactly one title; every copy always had one of three statuses (available, booked, borrowed); and every registered title carried both a stock counter and a waiting list. Together these clauses ensured that all entities were well-formed and that copies were consistently linked to titles and statuses.

Beyond basic typing, the invariant imposed a per-title capacity cap (each title's stock was bounded by the fixed limit, 4); a per-member quota that counted both copies held (booked or borrowed) and the number of distinct titles the member was queued for (thus was kept less or equal to the borrow limit, 2), so that future allocations remained feasible; required well-formed waiting lists per title (defined for every title, containing only registered members, with no duplicates) to preserve FIFO

discipline; and typed the ownership relation to cover both reservations and loans while remaining within member/copy domains.

3.7 INITIALISATION

An empty library was initialised: no titles, copies, members, or admins; catalogue, ownership, inventory, and waiting structures were empty, and the per-copy status map was initialised to mark all abstract BOOK values as available.

3.8 OPERATIONS

Add Title (addBookName): Inputs: a title to delete and an administrator. The operation was allowed only to an `admin` and only when the title did not already exist. After execution, the title was recorded with stock set to zero and an empty waiting list.

Delete Title (deleteBookName): Inputs: a title to delete and an administrator. Deletion was allowed only to an `admin` and only when the title existed, its stock was zero, no copy was catalogued to it, no member owned it, and its waiting list was empty. After deletion, the title and its related records were removed.

Add Copy (incInventory): Inputs: a new copy, a title and an administrator. The action was permitted only when the copy was not yet in the library, the target was an already registered title, the copy–title pair did not yet exist in the catalogue, the title’s stock was below the capacity limit, and the caller was an `admin`. After execution, the copy was added and linked to the title and the title’s stock increased by one. If the title’s waiting list was non-empty, the new copy was immediately booked for the head and the head was removed from the list; the “under-limit” test for the head was applied as a precaution, because joining a waiting list already consumed quota and the head was normally eligible unless their status had changed. If the head was ineligible, the copy remained available and the queue was left unchanged. When the waiting list was empty, the copy remained available.

Remove Copy (decInventory): Inputs: a copy to remove, a title and an administrator. Removal was permitted only when the copy was catalogued to that title, the title was registered, the title’s stock was positive, and the copy had no holder (neither `booked` nor `borrowed`); the caller had to be an `admin`. After execution, the copy and its catalogue link were removed and the title’s stock decreased by one. The mere presence of a waiting list imposed no additional restriction, since an available copy would have been immediately reserved for the head.

Add Member (addMember): Inputs: a person to register and an administrator. Registration was allowed only when the person was not already a member and the caller was an `admin`. After execution, the person was added to the set of members.

Delete Member (deleteMember): Inputs: a member to delete and an administrator. Deletion was allowed only to an `admin` and only when the person was currently registered as a member, held no copy in any state (neither `booked` nor `borrowed`), and did not appear in any waiting list. After deletion, the person was removed from the member set.

Add Administrator (addAdmin): Inputs: a person to add as an administrator. Creation was allowed when the person belonged to `ADMIN` and was not already in the `admin` set; within this model’s scope, no higher-privilege approval was required. After execution, the person appeared in the admin set.

Delete Administrator (deleteAdmin): Inputs: an administrator to remove. Removal was allowed when the person was currently in the `admin` set; within this model's scope, no higher-privilege approval was required. After execution, the person no longer appeared in the `admin` set.

Reserve Copy (reserveBook): Inputs: a copy, a title and a member. Reservation was allowed when the copy was in the library and catalogued to the title, the title existed and had positive stock, the copy's status was `available`, and the member was under the `borrow limit` (counting reserved books, borrowed books and distinct queued titles). After execution, the copy was marked booked for that member, the member-copy link was recorded, and the title's stock was unchanged.

Withdraw Reservation (withdrawReserveBook): Inputs: a copy, a title and the member who holds the reservation. Withdrawal was allowed when the member was registered, the copy existed and was catalogued to the title, the title existed and had positive stock, and the copy was currently `booked` for that member. After withdrawal, the copy became `available` and the ownership link was removed; if the title's waiting list was non-empty, the copy was immediately `booked` for the head and the head was removed (the under-limit check was treated as a precaution, since quota had already been consumed at join); otherwise, the copy remained available and the queue was unchanged. Stock was unchanged.

Borrow Copy (borrowBook): Inputs: a copy, a title and a member. Borrowing was allowed when the title had positive stock and the copy was catalogued to that title: either (i) from an `available` copy with the member under the borrow limit (counting reserved books, borrowed books and distinct queued titles), or (ii) from a copy already `booked` for the same member. After borrowing, the copy was set to `borrowed` and the title's stock was decreased by one; in case (i), the member-copy link was added, while in case (ii), the existing link was reused (no duplicate ownership was created).

Return Copy (returnBook): Inputs: a copy, a title and the member who currently holds it. Returning was allowed only for that member. After return, the copy became available, the title's stock increased by one, and, if the title's waiting list was non-empty, the copy was immediately `booked` for the head and the head was removed (the under-limit check was treated as a precaution, since the `borrow limit` had already been consumed at join).

Join Waiting List (addWaiting): Inputs: a title and a member. Joining was allowed for a registered member when the title existed, no copy of that title was `available`, the member was under the `borrow limit` (counting reserved books, borrowed books and distinct queued titles), and the member was neither already queued for that title nor already holding a copy of it. After joining, the member was appended to the end of the title's waiting list (FIFO), and the waiting slot counted towards the member's `borrow limit`.

Leave Waiting List (removeWaiting): Inputs: a title and a member. Leaving was allowed when the person was a registered member and appeared on the waiting list. After leaving, all occurrences of that member were removed from the sequence while preserving the order of the remaining entries; the waiting list remained in place, and inventory and copy statuses were unchanged.

3.9 Verification and Outcomes

3.9.1 Model checking (consistency & deadlock)

Method: Prob's state-space model checker was applied repeatedly during model construction; after each change, a full exploration over the finite domains stated earlier was run.

Use during model construction: When an invariant violation was reported, inspecting the shortest counterexample trace and the recorded sequence of operations was found to be effective for locating the faulty guard or post-condition. Because `NAT` was used in the model, `MAXINT` had to be set strictly above 4 in ProB; otherwise, spurious violations appeared where stock updates exceeded the representable range. Apparent deadlocks were sometimes observed in partial builds (e.g., after adding the `addBookName` operation without its complementary removal); such cases disappeared once the corresponding operations were added and were treated as work-in-progress artefacts rather than defects.

Outcome: On the completed model, model checking reported no invariant violations and no reachable deadlocks.

3.9.2 Assertions

Method: State assertions were stated as total-state predicates and were checked by ProB over all reachable states.

A1 – Inventory accounting. For every title, inventory equalled the number of copies whose status was `available` or `booked`. This captured the chosen semantics that reservations did not consume inventory and were preserved by `add/remove/borrow/return` operations.

A1b – Conservation. For every title, the number of `borrowed` copies equalled total copies minus inventory. Together with A1, this ensured consistent inventory accounting across all operations.

A2 – Exclusivity of non-available copies. Any copy in `booked` or `borrowed` had exactly one holder. This prevented double booking and matched the guards of `booked/borrowed`.

A3 – No owner for available copies. Any available copy had no holder. This ensured that ownership links only arise after booking and kept the catalogue consistent.

A4 – Ownership typing (members & copies). Ownership involved only registered members and existing copies. Early runs revealed a path where an ownership pair could survive a deletion step; this was fixed by (i) promoting the typing condition into the invariant and (ii) strengthening deletion preconditions (e.g., member deletion required no holdings; copy removal required the copy to be neither reserved nor borrowed).

A5 – Waiting lists contain only members. Each waiting list contained only registered members; this provided type safety for queue manipulation and held trivially once membership checks were enforced at join.

A6 – Holder not still queued for the same title. If a member held a copy of a title (`booked` or `borrowed`), the member did not appear in that title's waiting list. A6 exposed an early issue where dequeuing after allocation could be skipped; the post-conditions of `incInventory`, `returnBook`, and `withdrawReserveBook` were adjusted to remove the head on successful assignment, and a matching invariant was added to prevent re-queuing while holding.

Outcome: All assertions were satisfied on the completed model. In practice, A4 and A6 proved especially useful during construction: they pinpointed missing guards and post-conditions, which were corrected and eliminated the counterexamples. Assertions thus served as precise, executable documentation of the intended safety properties and contributed directly to defect removal.

3.9.3 LTL properties

LTL1 – Borrow-limit safety (global). Globally, each member's number of held copies was kept $\leq$ `borrow_limit`. (A restatement of the limit discipline to guard against regressions.)

LTL2 – Available copies are never owned (global). Globally, an `available` copy never appeared in the ownership relation.

LTL3 – Catalogue gluing (global). Globally, the catalogue's domain equalled the set of copies and its range was contained in the set of titles.

LTL4 – Reserve causes a status change (event). Whenever `reserveBook` occurred, at least one copy that was `available` became `booked`.

LTL5 – Borrow updates status/stock/ownership correctly (event). Whenever `borrowBook` occurred, the borrowed copy became `borrowed`, the corresponding title's stock decreased by one, and ownership was updated: if the copy had been `available`, the `(member→copy)` link was added; if it had been `booked` for the same member, the existing link was reused (no duplicate).

LTL6 – Return increases stock (event). Whenever `returnBook` occurred, the title's stock increased by one, and the copy's status became either `available` or `booked` (depending on possible re-allocation).

LTL7a – Add-copy auto-assignment (event, pre-state eligibility). Whenever `incInventory` occurred, if the title had an eligible member in the pre-state waiting list (head under the limit), the newly added copy was `booked` in the after-state.

LTL7b – Return auto-assignment + stock rule (event, pre-state eligibility). Whenever `returnBook` occurred, if the title had an eligible waiting member in the pre-state, the returned copy was `booked` in the after-state and the title's stock equalled pre-state stock + 1.

LTL7c – Withdraw keeps allocation when demand exists (event, pre-state eligibility). Whenever `withdrawReserveBook` occurred, if there was an eligible waiting member in the pre-state, the copy remained `booked` in the after-state (possibly for a different member); stock was unchanged.

Outcome: All LTL properties were satisfied on the completed model. During construction, these properties helped detect and correct early issues—for example, an initial `returnBook` variant that failed to increment stock (caught by LTL6/7b) and a `borrowBook` variant that duplicated ownership on the `booked→borrowed` path (caught by LTL5). The final checks confirmed the intended event semantics and global safety conditions.

4. Conclusion and Limitation

A B-Method specification of a single-branch Library Management System was produced and was analysed with ProB. Titles and copies were separated; per-copy status and per-title inventory were modelled distinctly; FIFO waiting lists were maintained; and state-changing actions were restricted to administrators. Under finite domains and fixed bounds (`limit=4`, `borrow_limit=2`), full model checking reported no invariant violations and no reachable deadlocks. Assertions A1–A6 were satisfied, giving evidence for correct stock accounting (A1/A1b), exclusivity/absence of owners (A2/A3), typing discipline (A4/A5), and the elimination of cases where a member both holds a copy and remains in the queue (A6). LTL properties were satisfied as well, confirming global safety and the intended post-conditions of reservation, borrowing, returning, and inventory updates. During construction,

A4/A6 identified missing pre-/post-conditions, and LTL5/LTL6/7b exposed early bugs (duplicate ownership; missing stock increment), which were corrected.

Limits were recognised: the scope excluded due dates, renewals, fines, notifications, multi-branch logistics, and higher-privilege admin control; domains were bounded for analysis; and real-time aspects and guarantees that everyone eventually gets a turn were not specified.